Light City

Design Document

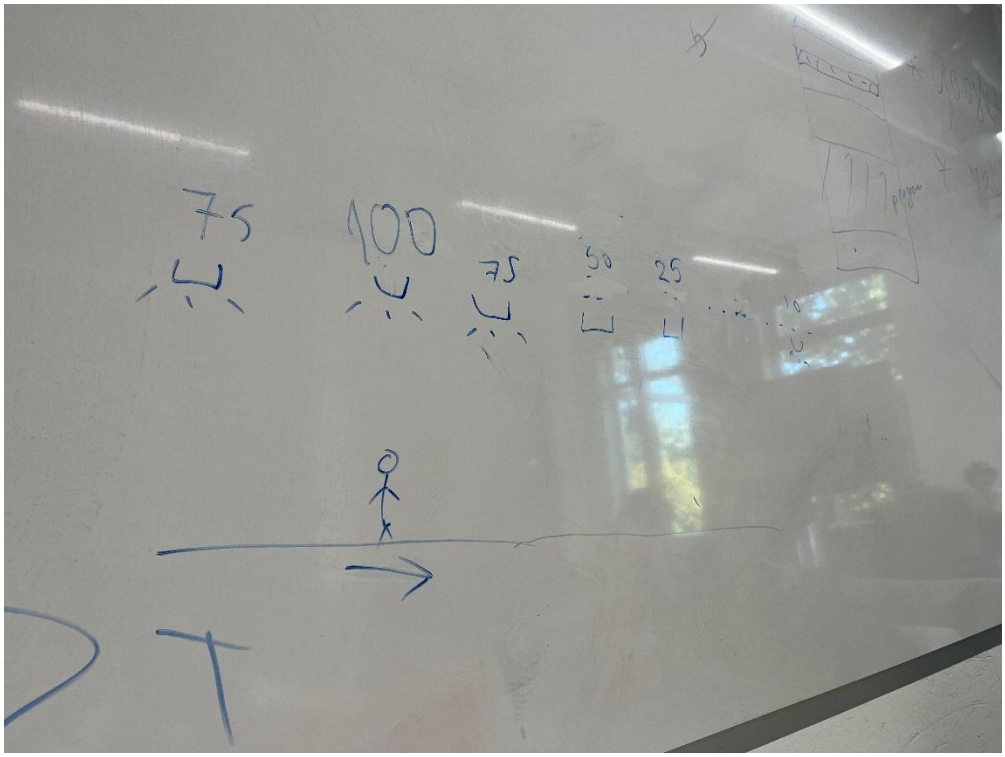
Version 1.3

Table of Contents

Version Overview

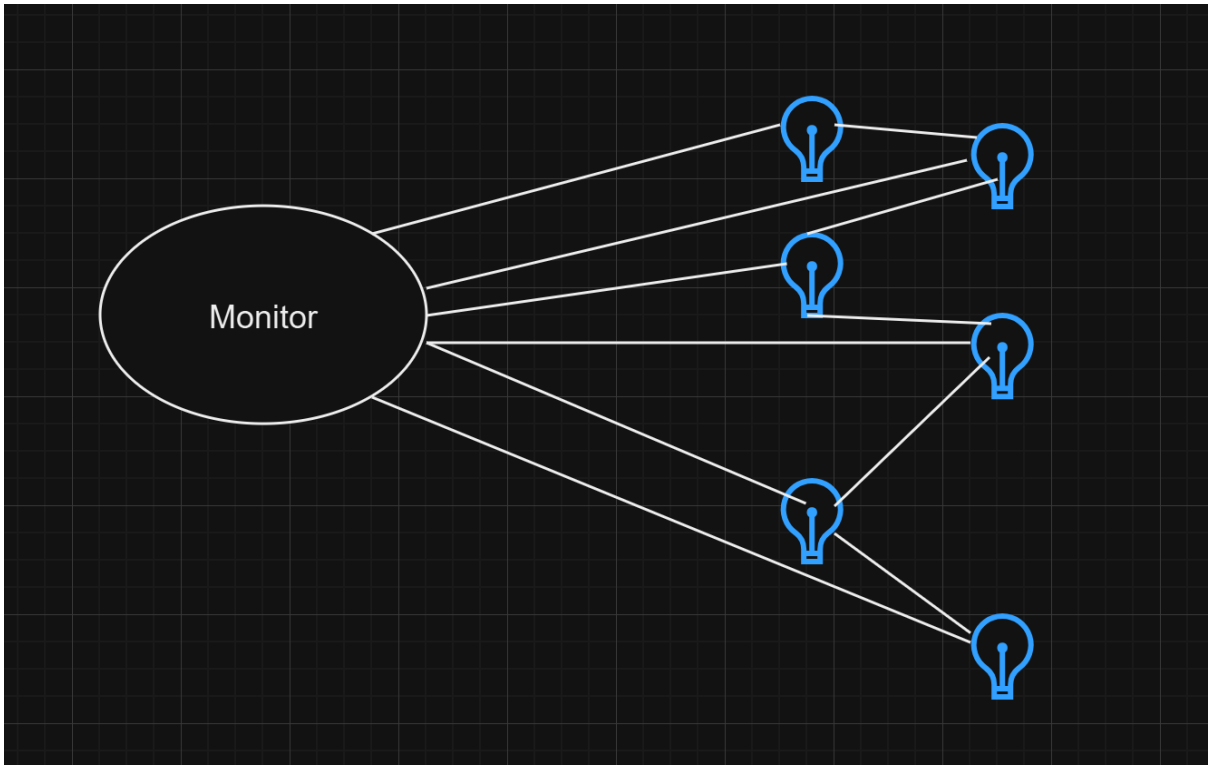
Version	Date	Name(s)	Summary
1.0	07/10/24	Veselin Atanasov	Added testing strategy
1.1	08/10/24	Veselin Atanasov	Expanded testing strategy
1.2	13/10/24	Marco Jacobs	Added the system design overview
1.3	11/11/24	Veselin Atanasov, Feng Wu	Updated protocol messages and add state diagram with state explanations
1.4	15/12/24	Veselin Atanasov	Updated message protocol for communication of nodes.
1.5	14/01/25	Marco Jacobs	Added the Class Diagram for the Monitor

Interaction With The Environment



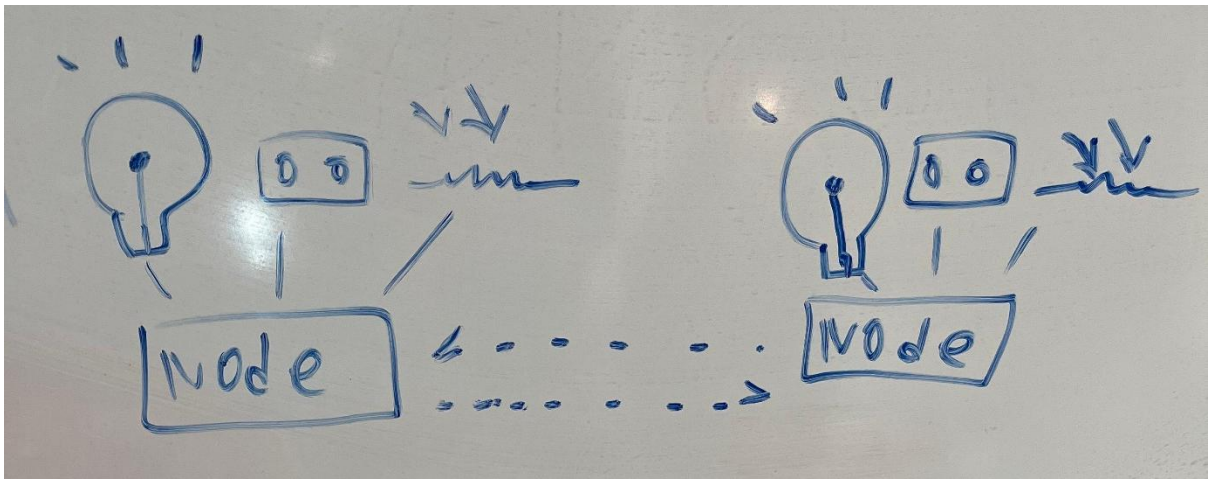
A person walking on the pavement, when a person is exactly below a light, the light above them shines at 100% intensity, and the lights after them gradually dims to 75%-50%-25%-10%. Neighboring nodes will adjust accordingly. Each light nodes should be able to know if neighboring nodes are functioning properly.

System Structure



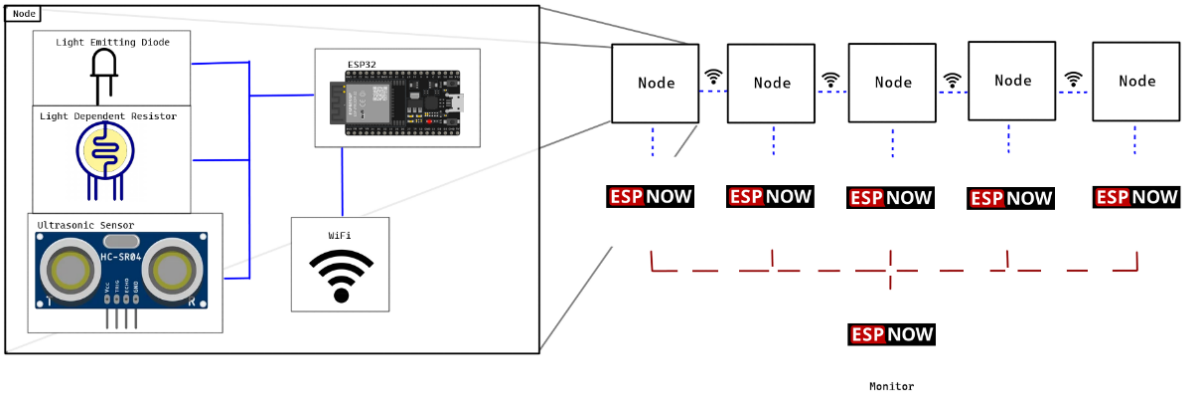
Nodes are connected to a monitor and each nodes are continuously communicating with each other to check if everything is working properly and if there is something that triggers the sensors. The nodes are not reliant on the monitor to operate correctly.

System Interactions



Each nodes will be communicating with each other wirelessly and each components of the nodes are going to be connected by wire. ESP32s are going to be used for the nodes.

System Design Overview

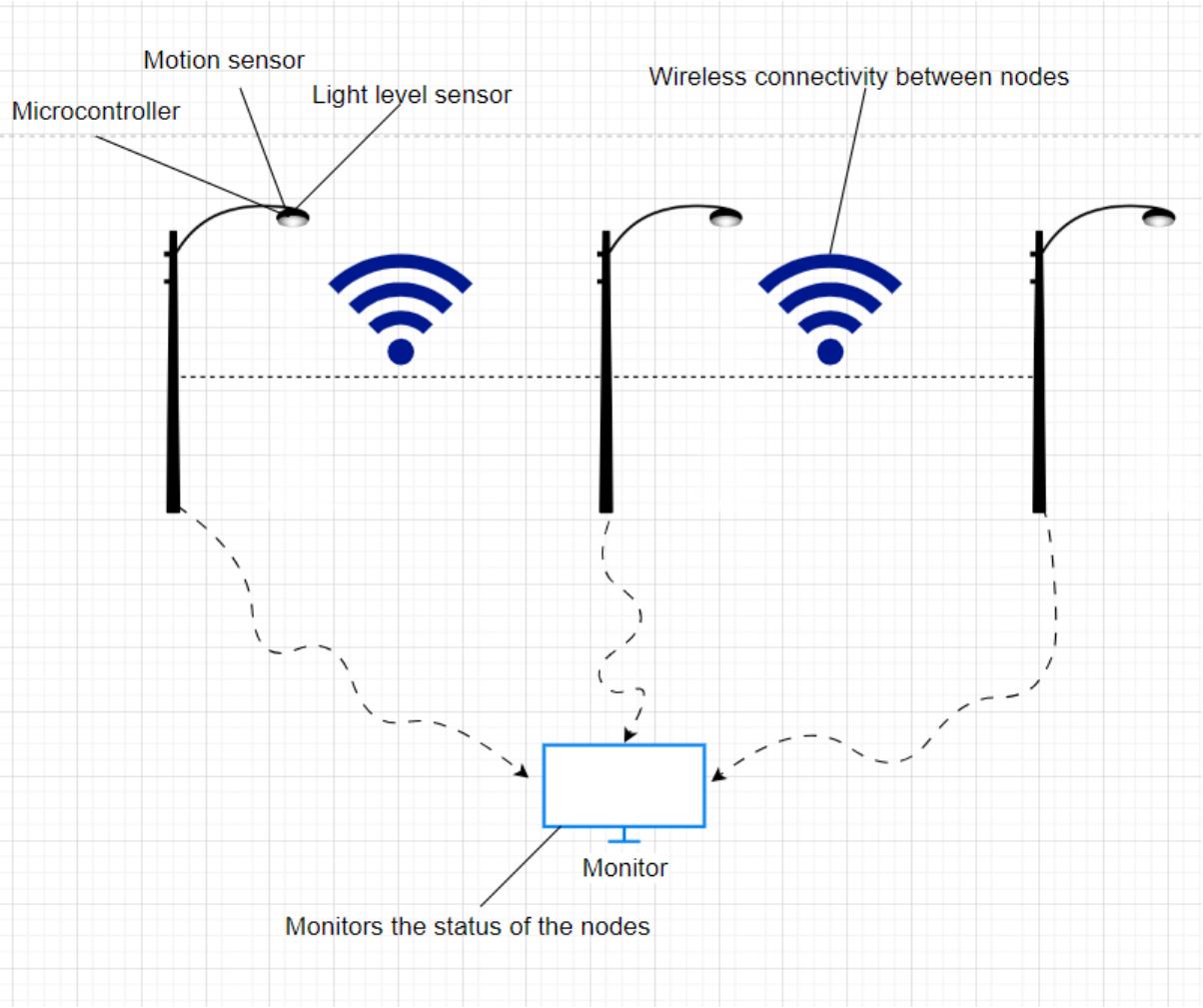


The system makes use of multiple nodes and they all consist of:

- An ESP32
- Ultrasonic Sensor
- LDR (Light Dependent Resistor)
- LED (Light Emitting Diode)

The nodes will communicate over ESP Now with the monitor.

Abstract System Design



Message Protocol

The messages that the system will use for communication between nodes will be listed and explained in this chapter. The messages that will be sent between nodes will be in struct format and will follow the following structure.

Message type as an enumeration. There is a few enums to describe different types of messages to handle all situations. The types of messages are ERR, LDR and PWR.

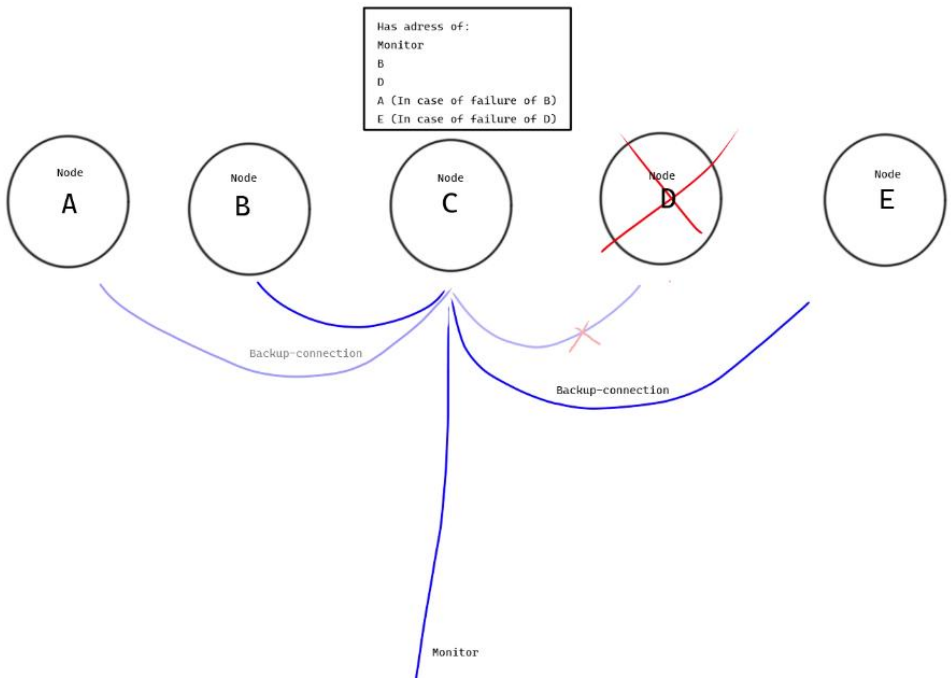
Message type	Description
ERR	Whenever a node fails, it sends a message to the monitor with the data to specify the error. The data

	of the ERR message type is also an enum. This enum has 3 members: - LEFT_ERROR_CODE-this message is sent to the monitor when this node fails to communicate with the left neighbour node. - RIGHT_ERROR_CODE-this message is sent to the monitor when communication with right neighbour fails. -HARDWARE_ERROR_CODE-this message is sent to the monitor when the light on the node fails.
LDR	This type of message sends the reading of the ldr in lux as data. This message is sent whenever a ping operation is done.
PWR	This message contains the duty cycle as % whenever the node detects a person, or it received the PWR message from another node. This message is sent to neighbor nodes or the nodes after the neighbors in case of communication error.

Communication Protocols

ESP Now, has been chosen due to it's flexibility. It is able to connect to multiple nodes at once and could implement a full duplex communication. The features of the protocol is sufficient for the project. ESP Now also provides connect/disconnect notification.

This protocol is used by having each nodes to have the MAC addresses of the neighbouring nodes, and they are set to communicate with each other, 1 node can both receive from multiple nodes and send to multiple nodes.



State Diagram

Depending on the lux that gets received by the LDR it will either goes to Day or Night state. During the Day state it will first send out a Ping() to ensure that connections between ESPs is still established. Afterwards with ldr.Read() it will read the value of lux, if it is above 400 (meaning it is daytime) it will go back to the Day state and go into deep sleep mode and it will keep checking every 30 minutes. Lastly, before it enters the deep sleep mode it will read the lights volt output. If the voltage is high, that means the LED/Lightbulb is not working. In that case it will send an error message.

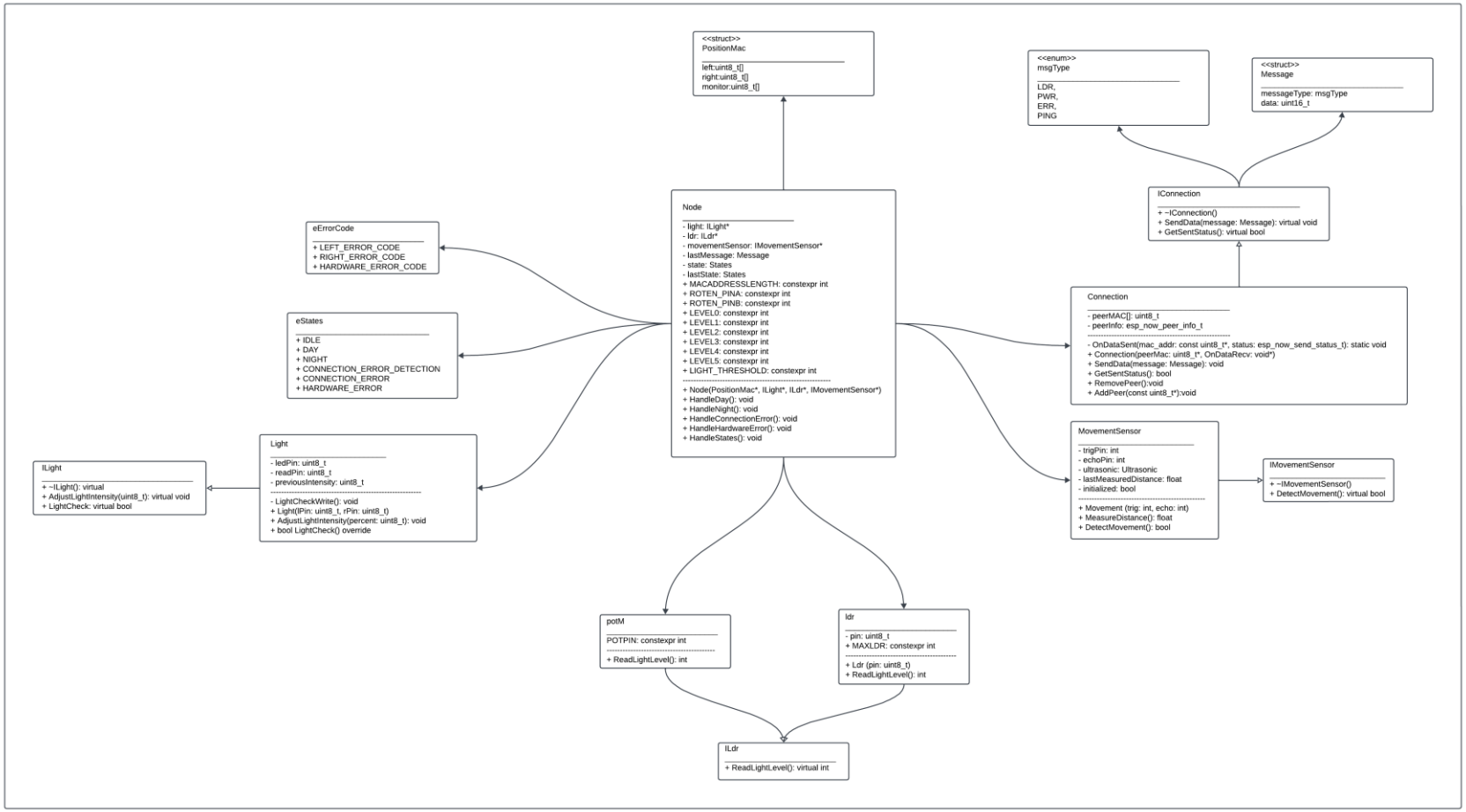
Alternatively, if it goes into the Night state. This state has a few functions that will be triggered to constantly check if any pedestrians are nearby, to immediately set the brightness to the correct level, to make sure the nodes are still connected, no broken LEDs/LDR/connection between the nodes.

Test Strategy

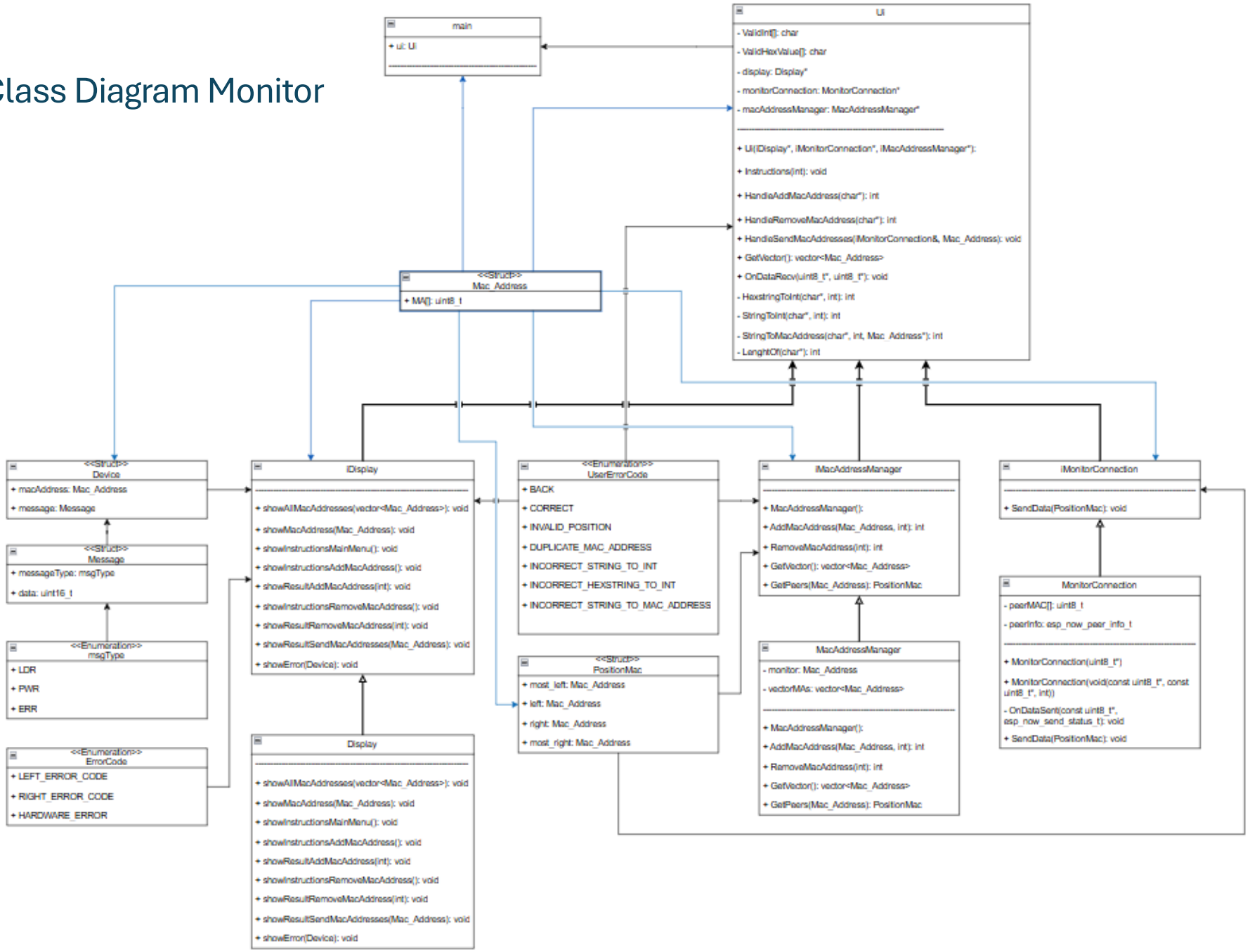
The test approach is to test the scenarios in a maximally accurate environment. That means putting the scenarios to the test in a realistic environment to see if the system behaves as expected. For testing purposes, a system of 5 nodes(lights) will be set up.

The first step of testing is to establish a connection between nodes and test it by sending messages that simulate the data of our application. Afterwards, code for each individual hardware component will be tested to ensure normal and predictable operation of the hardware components. The final step is to integrate all the subsystems together and troubleshoot any unexpected behaviour.

Class Diagram Node



Class Diagram Monitor



The monitor uses three different interface and multiple structs and enumerations. The MonitorConnection, Display and MacAddressesManager classes all have an interface.

The responsibilities are as follows:

- The Ui class is responsible for the entire system based on the user its inputs.
- The Display class is responsible for the displaying of the outcome of the actions made by the user.
- The MonitorConnection class is responsible for the communication between the monitor and the nodes.
- The MacAddressesManager class is responsible for keeping track of all the node mac addresses and its position compared to the other nodes.